

使用 Codex 搭建生产级应用

完整工作流与实战技巧

从需求澄清、仓库调查、设计与计划，到测试、评审、发布、回滚和线上反馈的技术栈中立方法。

版本：2026-07-10

适用对象：开发者、技术负责人和小型工程团队

原则：让每个重要结论都有证据，让每个高风险动作都有边界。

目录

按生产交付生命周期组织；附录提供可以直接复用的模板与检查清单。

阅读指南	3
1. 生产级交付不是“让 AI 写完代码”	4
2. 四个控制变量：上下文、约束、反馈、验证	4
3. 根据风险选择流程强度	5
4. 阶段一：定义目标、边界与成功条件	6
5. 阶段二：准备仓库、环境与持久指令	7
6. 阶段三：调查现状并建立证据	8
7. 阶段四：比较方案并确认设计	9
8. 阶段五：制定可执行计划	11
9. 阶段六：用测试和检查约束实现	12
10. 阶段七：小批次修改与持续验证	13
11. 阶段八：系统调试、回归与浏览器验收	14
12. 阶段九：安全、数据、性能与可运维性审查	16
13. 阶段十：代码评审与合并决策	17
14. 阶段十一：CI/CD、发布、迁移与回滚	18
15. 阶段十二：监控、事故反馈与知识沉淀	19
16. Codex 工作表面应该如何选择	21
17. 高效使用 Codex 的工作技巧	23
18. 高频失败模式与纠偏方法	25
19. 可复制的提示词模板	26
附录 A：一页式端到端工作流	29
附录 B：生产就绪检查清单	31
附录 C：AGENTS.md 通用模板	34
附录 D：能力选择速查表	35
附录 E：术语与官方参考资料	36

阅读指南

这不是一本 Codex 功能菜单，也不是一套“万能提示词”。它要解决的是更具体的问题：怎样把一个模糊需求，经过调查、设计、计划、实现、验证、评审和发布，变成可以承担真实用户、真实数据和真实故障成本的应用。

本手册把内容分成两类：

- Codex 产品事实：关于 [AGENTS.md](#)、项目配置、Skill、Plugin、MCP、Hook、Automation 以及不同 Codex 工作表面的说明，以 OpenAI 官方 Codex 文档和当前可用产品能力为依据。产品会持续更新，实际使用时应以文末官方链接为准。
- 软件工程建议：关于任务拆分、测试、评审、安全、迁移、发布和回滚的做法，是面向生产交付的通用工程总结，不代表 OpenAI 对某个技术栈的唯一推荐。

建议按两种方式使用：第一次从头读完，建立完整闭环；真正开发时，打开附录 A 的一页式流程和附录 B 的生产就绪检查清单，把它们当作每个功能的操作面板。

手册中“生产级”不是“用了微服务、Kubernetes 和复杂平台”，而是指：需求边界明确，关键风险被识别，变更可审查，结果有证据，数据有保护，故障可观测，发布可控制，必要时能回滚。

1. 生产级交付不是“让 AI 写完代码”

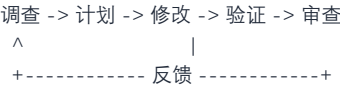
Codex 可以很快阅读代码、生成实现、运行命令、修改文件和解释错误，但“完成代码”与“完成交付”之间仍隔着一整套责任链。一个功能可能编译成功，却在权限、并发、数据迁移、兼容性、审计或回滚上不合格；一个缺陷可能暂时消失，却因为只处理了症状而在另一条调用路径中继续存在。

生产交付至少需要回答七个问题：

- 1. 做什么？ 用户问题、业务目标和成功条件是什么？
- 2. 不做什么？ 范围边界、非目标和明确禁止项是什么？
- 3. 在哪里改？ 真实调用链、数据流、依赖和现有约定是什么？
- 4. 为什么这样改？ 选择这个方案的证据和取舍是什么？
- 5. 如何证明？ 哪些测试、构建、静态检查、浏览器操作或人工验收能够证明结果？
- 6. 出了问题怎么办？ 日志、指标、告警、降级、迁移和回滚路径是什么？
- 7. 谁负责最终决定？ 谁接受安全、成本、合规、合并和上线风险？

Codex 适合承担高强度的信息处理和执行工作：探索仓库、定位调用关系、比较实现、编写小批次变更、生成测试、运行验证、整理评审结果。人类必须保留不可委托的决策：产品意图、风险接受、生产凭据、破坏性操作、合规判断、最终合并和发布授权。

最小可信闭环不是“提示词 -> 代码”，而是：



任何一步缺失都会制造典型风险：没有调查会改错位置，没有计划会造成大而散的变更，没有验证会把推测当事实，没有审查会漏掉测试没有覆盖的业务和安全问题。

人与 Codex 的责任边界

事项	Codex 可以做	人类必须决定
需求	整理歧义、提出验收标准	真实业务目标与优先级
架构	比较方案、标出依赖和风险	接受哪组取舍与长期成本
实现	修改代码、配置、文档和测试	是否允许越过范围或引入新依赖
安全	扫描常见问题、构造攻击场景	风险是否可接受、是否满足合规
数据	分析迁移、生成校验和回滚步骤	是否对生产数据执行操作
发布	生成发布清单、验证命令和说明	何时上线、是否继续或回滚

核心原则：让 Codex 执行可验证的工作，让人类承担不可外包的责任。

2. 四个控制变量：上下文、约束、反馈、验证

Codex 的输出质量并不只由提示词文采决定，而主要由四个工程变量决定。

2.1 上下文：给足决策所需信息，但不要倾倒整个世界

有效上下文包括目标、相关文件、现有约定、失败日志、可运行命令和已知限制。低质量上下文有两种极端：只说“帮我做登录”，信息不足；或者把整个仓库、全部日志和几十页设计一次塞入，让重要信号淹没在噪声中。

上下文应按任务逐层展开：先让 Codex 找到相关入口，再读取调用链和测试，再补充业务规则。把仓库中长期有效的内容放进 `AGENTS.md`，把一次性限制留在当前提示词，不要每轮重复传递。

2.2 约束：明确允许做什么，以及不能做什么

约束至少应包含范围、兼容性、依赖策略、安全边界、禁止命令和交付格式。例如：“只修改导出模块及其测试；不升级依赖；保持 API 响应结构；不得连接生产数据库；完成后运行以下三条命令。”

好的约束不是干涉实现细节，而是保护不可破坏的边界。如果你要求 Codex 必须使用某个函数名，却没有说明业务兼容性，可能是在约束错误的东西；如果你没有禁止修改迁移历史和生产密钥，才是真正缺少约束。

2.3 反馈：让每一小步都能暴露错误

反馈来自测试、编译器、类型检查、静态分析、日志、浏览器、数据库校验、代码评审和真实用户。任务越小，反馈越快，Codex 越容易定位因果关系。

不要等十几个文件都改完才运行测试。高效节奏是：先运行最小相关检查，修改一组紧密相关文件，再运行同一检查；阶段完成后运行更广的回归验证。

2.4 验证：用外部证据替代模型自信

“已经修复”“应该可以”“测试看起来会通过”都不是证据。证据必须来自刚刚执行的完整命令或可观察操作，例如：

- 指定测试用例从失败变为通过；
- 完整测试套件显示 0 failures；
- 构建命令退出码为 0；
- 浏览器实际走通关键流程并核对界面状态；
- 迁移在生产快照副本上前进和回滚成功；
- 安全扫描结果经过人工分类，无未处理高危项。

测试通过也不能自动证明需求满足。最终验收必须同时核对：实现是否符合需求、测试是否覆盖关键风险、验证是否足够完整、变更是否可运维和可回滚。

3. 根据风险选择流程强度

生产流程不应对所有任务一视同仁。修正文案与修改支付幂等逻辑需要不同强度。评估四个维度：影响范围、不可逆性、不确定性和外部影响。

风险级别	典型任务	计划要求	验证要求	人工门禁
低	文案、局部样式、内部脚本小修	简短步骤和范围	相关测试或可运行检查	合并前快速审阅
中	跨模块功能、公共 API、依赖升级	设计比较、明确文件与回归面	单元、集成、构建或浏览器验证	方案和合并审批
高	认证授权、支付、隐私、数据迁移、发布系统	正式设计、威胁与回滚分析、分阶段计划	完整测试、数据演练、安全与发布验证	多人审批、生产操作确认

风险升级信号

遇到下列任一项，应提高流程强度：

- 改动公共协议、数据库结构或跨服务契约；
- 涉及身份、权限、资金、个人信息或密钥；
- 故障可能造成数据丢失、重复操作或长时间中断；

- 依赖外部系统，且无法在本地完整模拟；
- 需要执行不可逆命令或生产写操作；
- 团队无法快速理解和审查变更；
- 没有可靠测试，或者历史上该区域经常回归。

三档执行模式

快速模式：调查 -> 小改 -> 相关验证 -> 审阅

标准模式：调查 -> 设计 -> 计划 -> 测试约束 -> 分批实现 -> 全面验证 -> 评审

高风险模式：标准模式 + 威胁建模 + 数据/发布演练 + 双人门禁 + 回滚验证

不要用“这是 AI 写的”作为提高或降低风险的理由。流程强度由变更本身的后果决定。

4. 阶段一：定义目标、边界与成功条件

目标

把“想做一个功能”转换成可以被实现和验收的任务合同。结束时，团队应对用户价值、范围、非目标、约束和验收标准达成一致。

给 Codex 的输入

- 原始需求、用户场景和业务背景；
- 已知限制，例如兼容版本、时间窗口和法规要求；
- 当前系统行为、期望行为和可复现示例；
- 明确的非目标与不可修改区域；
- 需要保留的接口、数据或用户体验。

Codex 应执行的动作

1. 用自己的话复述目标，区分事实、假设和未知项；
2. 找出会改变实现方向的歧义，而不是询问无关偏好；
3. 把需求写成可观察的验收标准；
4. 列出正常路径、边界条件、错误路径和权限场景；
5. 标出可能需要安全、数据或发布专项审查的部分；
6. 在获得确认前不进入实现。

人类决策点

- 哪个用户问题最重要；
- 这次交付包含和不包含什么；
- 哪些兼容性和风险不可接受；
- 验收标准是否真实代表业务成功。

必须形成的产物

- 一句话目标；
- 范围与非目标列表；
- 可验证的验收标准；

- 关键约束和风险初筛；
- 尚未解决的问题及负责人。

退出条件

所有会显著改变架构或工作量的歧义已经解决；每条验收标准都能通过测试、命令、浏览器操作或明确人工检查判断通过/失败；人类已经确认范围。

推荐提示词

目标：把下面的原始需求整理成可实施任务，但先不要写代码。

背景：[业务与用户场景]

原始需求：[需求文本]

范围：[本次必须包含]

非目标：[明确不做]

约束：[兼容性、安全、时间、依赖]

请输出：

1. 目标复述；
2. 事实、假设和未知项；
3. 最多 5 个会改变实现方向的问题；
4. Given/When/Then 或等价形式的验收标准；
5. 风险级别及理由。

在我确认前不要实现。

5. 阶段二：准备仓库、环境与持久指令

目标

让 Codex 在可重复、可隔离、权限适当的环境中工作，并知道仓库长期有效的规则。重点不是堆积说明，而是让它能找到正确命令、遵守现有模式、保护用户改动。

给 Codex 的输入

- 仓库根目录和受信任范围；
- 安装、启动、测试、构建、格式化和数据库命令；
- 代码风格、目录职责和架构边界；
- 密钥、生产系统、危险命令和外部写操作的限制；
- 当前工作区是否有未提交改动；
- 针对当前任务的特殊限制。

Codex 应执行的动作

1. 查找根目录及更深层级的 `AGENTS.md`，确认适用范围；
2. 检查仓库状态，不覆盖或清理不属于当前任务的改动；
3. 识别语言、包管理器、测试框架、构建工具和 CI 配置；
4. 优先使用项目已有工具和锁文件，不擅自更换依赖管理方式；
5. 运行只读或低风险的环境检查；
6. 在高风险修改前建议隔离分支或 worktree；
7. 对网络、生产、密钥、删除和外部写操作遵守审批边界。

人类决策点

- 仓库是否可信，允许多大范围的文件和命令访问；
- 哪些规则应长期写入 `AGENTS.md`，哪些只属于本次任务；
- 是否创建分支、worktree 或临时环境；
- 是否允许安装依赖、联网或连接外部服务。

必须形成的产物

- 可运行的命令清单；
- 适用的 `AGENTS.md` 或等价项目说明；
- 环境和权限边界记录；
- 基线测试结果；
- 已有未提交改动的保护说明。

退出条件

Codex 能明确说出仓库结构、正确命令、持久规则、当前变更状态和权限边界；基线检查已运行，或无法运行的原因已经记录并得到同意。

推荐提示词

先准备工作环境，不要修改业务代码。

请：

1. 查找并读取适用的 `AGENTS.md`；
2. 检查仓库状态，列出已有改动并承诺不覆盖；
3. 识别安装、测试、构建、格式化和启动命令；
4. 运行最小只读环境检查与基线测试；
5. 汇报权限、依赖或环境阻塞。

禁止：删除文件、重置 Git、升级依赖、访问生产系统、输出秘密。

输出：仓库地图、命令表、基线证据、风险和下一步建议。

6. 阶段三：调查现状并建立证据

目标

在提出方案之前理解真实系统：入口在哪里，数据如何流动，哪些调用者受影响，现有测试保护什么，问题能否复现。调查必须把“猜测”变成“带文件、调用链和命令的证据”。

给 Codex 的输入

- 任务目标和验收标准；
- 错误信息、日志、请求示例或用户操作步骤；
- 初步怀疑的模块，但不要把猜测当结论；
- 允许读取和运行的范围；
- 已知正常版本、失败版本或最近相关变更。

Codex 应执行的动作

1. 先用文件和文本搜索定位入口、符号、路由、配置和测试；

2. 沿调用者、被调用者、数据模型和外部依赖追踪真实路径；
3. 阅读相邻实现，识别仓库已有模式和可复用工具；
4. 运行最小复现或相关测试，记录完整命令和输出；
5. 区分观察到的事实、推断和待验证假设；
6. 对缺陷建立根因链，而不是看到第一个异常就修改；
7. 形成受影响范围和回归面。

人类决策点

- 是否允许扩大调查范围；
- 哪些日志、数据或外部系统可以安全访问；
- 当前证据是否足以进入方案设计；
- 无法复现时是否接受增加观测后再发布诊断版本。

必须形成的产物

- 相关文件和职责地图；
- 调用链或数据流说明；
- 复现步骤和失败证据；
- 根因假设及支持/反对证据；
- 受影响的接口、测试、数据和运维面。

退出条件

功能任务已经找到现有扩展点；缺陷任务能够稳定复现，或已明确说明为什么不能复现及下一步观测方案；关键结论可以指向代码、配置、日志或命令证据。

推荐提示词

请调查这个任务，不要修改文件。

目标：[目标]

已知现象：[日志/操作/输入输出]

范围：[允许调查的目录或服务]

请沿真实调用链回答：

1. 入口、核心逻辑、数据层和外部依赖分别在哪里；
2. 有哪些调用者和现有测试；
3. 当前行为如何复现；
4. 哪些是事实，哪些是假设；
5. 最可能根因和受影响范围是什么。

输出时给出文件路径、符号名、运行命令和关键结果。

7. 阶段四：比较方案并确认设计

目标

在实现前比较至少两个真正不同的可行方案，明确边界、数据流、错误处理、兼容性和测试策略。设计不是为了写长文，而是为了在代码成本发生之前暴露关键取舍。

给 Codex 的输入

- 已确认需求和调查结果；
- 现有架构、扩展点和约束；
- 性能、安全、数据和兼容性要求；
- 团队维护能力与交付时间；
- 明确的非目标。

Codex 应执行的动作

1. 提出 2 至 3 个可行方案，而不是一个方案的表面变体；
2. 对比复杂度、变更范围、运行成本、测试难度、回滚和长期维护；
3. 推荐最小且能满足当前需求的方案；
4. 说明组件职责、接口、数据流、失败路径和状态变化；
5. 对数据库、协议和外部集成给出兼容与迁移策略；
6. 为关键风险设计验证方法；
7. 等待人类确认后再写实施计划。

人类决策点

- 接受哪组复杂度、成本和扩展性取舍；
- 是否愿意承担迁移或兼容成本；
- 哪些风险需要原型、压测或安全评审；
- 设计是否足够小，可以在一个实施周期中完成。

必须形成的产物

- 方案对比表和推荐理由；
- 组件边界与数据流；
- 错误处理、兼容、迁移和回滚策略；
- 测试与观测设计；
- 经确认的设计说明。

退出条件

推荐方案被明确批准；主要组件都能回答“做什么、如何调用、依赖什么”；高风险状态变化有失败和回滚路径；没有影响方向的未决问题。

推荐提示词

基于已确认需求和调查结果，请比较 2-3 个可行设计，不要实现。

必须比较：变更范围、复杂度、兼容性、性能、安全、数据迁移、测试、可观测性、回滚和长期维护成本。

推荐满足当前需求的最小方案，并说明：

- 组件职责和接口；
- 数据与状态流；
- 错误和重试策略；
- 测试与发布验证；
- 已知限制和未来升级触发条件。

8. 阶段五：制定可执行计划

目标

把设计拆成一组可独立理解、可验证、可审查的小任务。计划应让一个不了解前文的工程师也能知道修改哪些文件、为什么修改、如何验证、何时停止。

给 Codex 的输入

- 经批准的设计说明；
- 当前文件结构、测试和构建命令；
- 交付优先级、风险等级和发布窗口；
- 团队的提交、评审和文档约定；
- 允许并行或必须串行的依赖关系。

Codex 应执行的动作

1. 先画出将创建和修改的文件及单一职责；
2. 把任务按依赖顺序排列，每个任务产生一个可运行增量；
3. 对非平凡逻辑安排“失败检查 -> 最小实现 -> 通过检查”的节奏；
4. 写明精确命令和期望结果，不使用“补充适当测试”等空话；
5. 在高风险节点插入人工确认和回滚检查；
6. 避免把无关重构、未来扩展和依赖升级夹带进计划；
7. 自审计划对设计的覆盖、占位符和命名一致性。

人类决策点

- 任务大小是否适合审查和回滚；
- 是否允许并行执行；
- 哪些节点必须停下来评审；
- 是否接受计划中的依赖、迁移和发布安排。

必须形成的产物

- 文件变更地图；
- 有序任务列表；
- 每个任务的实现和验证步骤；
- 人工门禁和回滚点；
- 需求到任务的覆盖关系。

退出条件

每条需求至少映射到一个任务；每个任务有明确文件、动作、命令和通过条件；没有 TBD、TODO 或“稍后处理”；计划经人类确认。

推荐提示词

请把已批准设计转换成可执行实施计划，不要修改代码。

要求：

- 先列文件结构与职责；
- 任务按依赖顺序，每个任务形成可运行增量；
- 每一步给出精确文件、动作、命令和期望结果；
- 非平凡逻辑使用失败检查 -> 最小实现 -> 通过检查；
- 标出安全、数据迁移、外部写操作和发布门禁；
- 不加入未请求重构、未来抽象或依赖升级；
- 最后逐条核对设计覆盖率。

9. 阶段六：用测试和检查约束实现

目标

在写实现之前定义可失败的行为检查，使 Codex 能围绕明确反馈工作，并证明测试确实能捕获目标问题。测试不是追求数量，而是保护关键行为和风险边界。

给 Codex 的输入

- 验收标准和设计中的风险；
- 现有测试框架、约定和相关用例；
- 缺陷复现输入或功能的行为示例；
- 外部依赖的可控测试替身；
- 性能、安全或迁移的专门验证要求。

Codex 应执行的动作

1. 选择最小且正确的测试层级：单元、集成、契约、端到端或运行检查；
2. 先写能表达行为的失败测试，运行并确认失败原因正确；
3. 缺陷修复时验证红绿循环：修复前失败，修复后通过；
4. 覆盖权限、边界、错误路径和重要状态变化，而非只测快乐路径；
5. 避免过度 mock 实现细节，使测试在错误实现下也可能通过；
6. 对无法自动化的项目写清人工验收步骤和证据；
7. 控制测试规模，优先一个能阻止回归的强测试。

人类决策点

- 哪些行为最值得长期保护；
- 哪些外部依赖需要真实集成环境；
- 测试成本与风险是否匹配；
- 无法自动化的验收是否可以接受。

必须形成的产物

- 失败测试或可运行检查；
- 修复前失败证据；
- 覆盖的行为与未覆盖风险说明；

- 人工验收步骤（如需要）。

退出条件

测试在实现前因预期原因失败；不会因为环境损坏、语法错误或测试自身错误而产生假红；关键验收标准已映射到自动或人工检查。

推荐提示词

先为这个行为创建最小失败检查，不要实现功能。

验收标准：[标准]

风险边界：[权限/并发/错误/数据]

现有测试入口：[命令或文件]

请选择正确测试层级，编写一个最小但有区分力的测试，运行它，并证明失败原因正是缺少目标行为。不要过度 mock，也不要顺便重构。

10. 阶段七：小批次修改与持续验证

目标

按计划完成最小实现，每次只引入一组容易理解的因果变化。通过频繁验证缩短错误发现距离，保护用户已有改动，并保持 diff 可审查。

给 Codex 的输入

- 当前计划任务和允许修改的文件；
- 失败测试、相关验证命令和完成定义；
- 代码风格、架构边界和兼容要求；
- 当前工作区状态；
- 明确禁止的重构、依赖和外部操作。

Codex 应执行的动作

1. 重新读取即将修改的文件和调用者，避免基于过期上下文编辑；
2. 优先复用仓库已有模式、标准库和已安装依赖；
3. 写使当前失败检查通过的最小代码；
4. 保留输入验证、安全措施、错误处理和可观测性；
5. 修改后先运行最小相关检查，再运行扩大范围的检查；
6. 检查 diff，移除调试输出、无关格式化和意外文件；
7. 发现计划错误或范围扩大时停止，返回设计或计划，而不是悄悄偏航。

人类决策点

- 是否接受超出原计划的新发现；
- 是否批准新增依赖或改变公共接口；
- 是否允许对已有脆弱代码做针对性改善；
- 当前批次是否足够小，可以继续或回退。

必须形成的产物

- 聚焦的代码和测试变更；

- 当前批次的验证输出；
- 任何设计偏差及批准记录；
- 干净、可解释的 diff。

退出条件

当前任务的失败检查已通过；相关回归未出现；diff 中没有无关改动、秘密或调试残留；变更仍符合已批准设计和范围。

推荐提示词

只执行计划中的当前任务：[任务名称]

允许修改：[文件]

禁止：[升级依赖/修改公共协议/无关重构/清理用户改动]

通过条件：[测试和命令]

请先重新读取相关调用链，然后做最小实现。修改后依次运行相关测试、静态检查和 diff 检查。若必须扩大范围，停止并说明原因，不要自行继续。

11. 阶段八：系统调试、回归与浏览器验收

目标

在局部实现通过后，验证系统级行为，并对任何失败进行根因分析。对于用户界面和浏览器流程，必须观察真实渲染和交互，而不是只根据组件代码推断。

给 Codex 的输入

- 完整复现步骤、环境和预期结果；
- 最新构建和测试命令；
- 服务日志、网络请求、数据库状态和浏览器控制台信息；
- 关键用户路径与兼容浏览器/设备范围；
- 已知不稳定测试和环境限制。

Codex 应执行的动作

1. 先稳定复现，再提出修复；
2. 从边界到内部逐层收集证据：输入、接口、业务逻辑、数据、外部依赖和输出；
3. 一次验证一个假设，避免同时修改多个可能原因；
4. 找到共享根因和所有调用者，在正确边界修复一次；
5. 运行相关、模块级和全套回归测试；
6. 对 UI 实际打开页面，检查视觉、交互、错误态、响应式和无障碍基础；
7. 记录无法复现、偶发失败或环境差异，不用“重跑通过”掩盖不稳定性。

人类决策点

- 是否接受临时缓解还是必须根治；
- 哪些浏览器、设备和用户路径是发布门槛；
- 不稳定测试是阻塞发布还是登记修复；
- 是否需要真实外部系统或生产影子环境验证。

必须形成的产物

- 根因说明和证据链；
- 相关及完整回归结果；
- 浏览器操作记录或截图（适用时）；
- 已知限制和剩余风险。

退出条件

原始症状无法在修复版本上复现；回归测试没有未解释失败；UI 变更经过真实浏览器验证；任何剩余风险被明确接受或进入阻塞列表。

推荐提示词

请按系统化调试处理，不要先猜修复。

复现步骤：[步骤]

预期/实际：[对比]

证据：[日志、请求、截图、测试]

先建立稳定复现和调用链；列出按可能性排序的假设；一次只验证一个。

找到根因后，在所有调用者共享的正确边界做最小修复，并运行：

1. 原始复现；2. 相关测试；3. 模块回归；4. 完整验证。

UI 变更必须在浏览器实际验收。

12. 阶段九：安全、数据、性能与可运维性审查

目标

在功能正确之外，评估它是否能在恶意输入、真实数据量、部分故障和长期运行条件下安全工作。审查重点是信任边界和失败后果，而不是机械勾选工具输出。

给 Codex 的输入

- 数据分类、身份与权限模型；
- 外部入口、第三方服务和秘密管理方式；
- 数据模型、迁移和保留策略；
- 预期负载、延迟和资源限制；
- 日志、指标、追踪、告警和运行手册约定。

Codex 应执行的动作

1. 画出信任边界和敏感数据流；
2. 检查认证、授权、对象级越权、输入验证、输出编码和秘密泄露；
3. 检查重放、重复提交、并发竞争、幂等和事务边界；
4. 对迁移评估锁表、默认值、回填、双写/双读、回滚和备份；
5. 检查 N+1、无界查询、内存增长、超时、重试风暴和缓存一致性；
6. 确保错误可定位但不向用户泄露内部细节；
7. 为关键失败建立指标、告警、日志字段和运行手册；
8. 把扫描结果按真实可利用性和影响分类，不直接把工具数量当结论。

人类决策点

- 风险是否满足组织安全与合规要求；
- 是否需要安全专家、数据负责人或运维审批；
- 性能目标和容量余量是否可接受；
- 哪些风险必须修复，哪些可以带期限接受。

必须形成的产物

- 威胁和信任边界清单；
- 数据迁移、校验与回滚方案；
- 性能假设和验证结果；
- 日志、指标、告警和运行手册更新；
- 风险处置表。

退出条件

无未处理的高危安全或数据风险；迁移在代表性环境演练；关键性能假设经过测量；故障发生时有足够观测和明确响应路径。

推荐提示词

请对当前变更做生产风险审查，不要只列通用清单。

基于真实调用链和数据流检查：认证、授权、对象级越权、输入、输出、秘密、并发、幂等、事务、迁移、回滚、N+1、无界资源、超时重试、日志泄露、指标和告警。

每个发现输出：证据位置、攻击/故障场景、影响、严重度、修复建议、验证方法。区分已证实问题和需要进一步验证的假设。

13. 阶段十：代码评审与合并决策

目标

从需求、正确性、安全、维护性和发布风险多个角度审查变更。评审不是让 Codex 复述 diff，而是寻找会导致错误行为、数据损坏、权限绕过或维护困难的具体问题。

给 Codex 的输入

- 需求、设计和实施计划；
- 完整 diff 与相关上下文；
- 测试、构建、安全和浏览器验证结果；
- 代码所有权和评审标准；
- 已知限制、迁移和发布计划。

Codex 应执行的动作

1. 先核对需求覆盖，再检查实现细节；
2. 沿调用链检查边界、错误处理、状态变化和兼容性；
3. 验证测试是否真正覆盖风险，而非仅统计数量；
4. 以严重度排序发现，并给出精确文件和行为后果；
5. 区分必须修复、建议改进和无关偏好；
6. 检查 diff 中的秘密、生成物、调试代码和意外大文件；
7. 如果没有可操作问题，明确说明审查范围和剩余测试缺口。

人类决策点

- 哪些问题阻塞合并；
- 是否接受剩余风险和技术债；
- 变更是否需要额外领域专家；
- 是否满足团队定义的完成标准。

必须形成的产物

- 按严重度排序的评审发现；
- 每项发现的证据、后果和修复建议；
- 需求覆盖和测试缺口说明；
- 合并或返工决定。

退出条件

所有阻塞项已经修复并重新验证；无未解释高风险发现；评审者能够理解变更目的、范围、验证和回滚；合并获得授权。

推荐提示词

请把这次变更作为生产代码评审，不要做摘要式表扬。

依据：[需求/设计/计划]

验证证据：[命令和结果]

优先寻找会导致错误行为、权限问题、数据损坏、兼容破坏、并发缺陷、不可运维或测试盲区的问题。按严重程度输出，每项包含文件位置、触发场景、实际后果和建议修复。忽略纯风格偏好，除非它会造成真实维护风险。

14. 阶段十一：CI/CD、发布、迁移与回滚

目标

把“本地正确”转换成“可控制地进入生产”。发布必须具备可重复构建、环境校验、分阶段暴露、观测阈值、停止条件和经过验证的回滚路径。

给 Codex 的输入

- CI 流水线、制品和环境配置；
- 发布拓扑、流量策略和维护窗口；
- 数据迁移顺序和兼容窗口；
- 健康检查、关键指标和告警阈值；
- 回滚、降级、功能开关和负责人。

Codex 应执行的动作

1. 确认 CI 运行完整测试、静态检查、构建和必要安全扫描；
2. 验证构建制品可追溯、配置来自环境且不包含秘密；
3. 把数据库变化设计成向前/向后兼容的分阶段过程；
4. 先在代表性环境演练迁移、部署、健康检查和回滚；
5. 生成发布前、发布中和发布后清单；
6. 定义金丝雀/灰度范围、观测窗口、成功阈值和自动/人工停止条件；
7. 在任何生产写操作、密钥或破坏性命令前请求明确授权；
8. 保存实际发布证据和时间线。

人类决策点

- 是否批准生产变更；
- 何时扩大流量、暂停或回滚；
- 数据迁移风险是否被接受；
- 谁负责指挥、沟通和事故升级。

必须形成的产物

- 通过的 CI 和可追溯制品；

- 发布运行手册；
- 数据迁移与回滚演练结果；
- 观测指标、阈值和停止条件；
- 发布记录与负责人。

退出条件

所有发布门禁通过；回滚或降级路径实际演练；关键指标在观测窗口内正常；生产操作经授权；发布记录完整。

推荐提示词

请为当前变更生成生产发布运行手册，不要执行生产操作。

包含：CI 门禁、制品校验、配置与秘密检查、迁移顺序、兼容窗口、灰度步骤、健康检查、关键指标与阈值、扩大流量条件、停止条件、回滚/降级步骤、负责人和沟通模板。

所有生产写操作、密钥访问和破坏性命令必须标为“需人工明确授权”。

15. 阶段十二：监控、事故反馈与知识沉淀

目标

把上线后的真实信号重新送回工程流程。功能发布不是终点；只有在指标、错误、用户反馈和运维经验被观察并沉淀后，团队才形成可持续改进闭环。

给 Codex 的输入

- 发布基线和预期指标；
- 日志、指标、追踪、告警和用户反馈；
- 事故时间线和操作记录；
- 已知风险、技术债和后续观察窗口；
- 文档、运行手册和任务系统位置。

Codex 应执行的动作

1. 对比发布前后关键指标，识别异常而不把相关性直接当因果；
2. 聚合错误但保留版本、请求、租户和环境等定位维度；
3. 对事故整理无责时间线、触发条件、放大因素和防线缺口；
4. 把行动项写成有负责人、期限和验证方式的任務；
5. 更新 `AGENTS.md`、测试、运行手册、架构决策和提示词模板；
6. 删除已经失效或会误导后续工作的旧指令；
7. 对重复检查考虑自动化，但只自动化稳定、机械、可观测的步骤。

人类决策点

- 指标变化是否需要回滚或后续修复；
- 哪些事故行动项优先；
- 哪些知识值得成为持久规则；
- 哪些检查适合自动化，哪些必须保留人工判断。

必须形成的产物

- 发布后验证记录；
- 事故或异常分析；
- 有负责人和验证条件的行动项；
- 更新后的测试、文档和持久指令；
- 下一轮风险和改进清单。

退出条件

观察窗口结束且关键指标可接受；异常已解释或进入任务；事故行动项可追踪；新的工程知识已进入仓库可发现位置，而不是只留在聊天记录中。

推荐提示词

请分析这次发布后的信号并形成工程反馈闭环。

发布版本/时间：[信息]

基线与当前指标：[数据]

错误、告警和用户反馈：[证据]

区分事实、相关性和因果假设；整理异常影响与时间线；提出行动项，
每项包含负责人角色、优先级、期限、验证方式和应更新的测试/文档/规则。
不要把临时聊天结论当成持久知识。

16. Codex 工作表面应该如何选择

Codex 中最常见的配置错误不是“没有配置”，而是把信息放在错误的作用域：一次性要求写进全局规则，仓库约定只存在聊天里，机械安全门禁只靠提醒，外部实时数据依赖模型记忆。选择原则是：使用能够覆盖需求的最小持久表面。

16.1 作用域选择表

需求	推荐表面	适合放什么	不适合放什么
只影响一个任务或线程	当前提示词/线程上下文	本次范围、临时约束、验收标准	跨任务长期规则
仓库约定与验证命令	AGENTS.md	目录职责、命令、风格、安全边界、完成定义	秘密、经常变化的临时状态
受信任仓库的 Codex 设置	项目 .codex/config.toml	模型、沙箱、MCP、Hooks 等项目设置	业务需求说明
个人跨仓库默认值	全局配置/全局指导	个人工具偏好、通用默认行为	某个仓库特有规则
可复用任务流程	Skill	有步骤、参考资料或脚本的工作方法	只运行一次的简单提示词
可安装的能力包	Plugin	Skills、工具、MCP 配置、Hooks、资源等组合	单一小流程
实时外部数据或动作	MCP Server/Connector	GitHub、文档、工单、数据库或企业应用连接	可以从本地仓库直接获得的信息
工具调用前后的机械执行	Hook	命令阻断、审计、格式检查、生命周期规则	需要业务判断的模糊决策
定时检查与后续工作	Automation	监控、提醒、定期验证、线程跟进	当前立刻完成的一次性任务

以上映射来自 OpenAI 官方 Codex 自定义和配置体系。实际字段、可用性和管理员限制可能随版本、账户和工作区策略变化。

16.2 AGENTS.md：仓库的可执行说明书

AGENTS.md 适合记录稳定、可操作、与仓库一起演进的规则。更深目录中的文件可以为子树提供更具体的指导，因此大型仓库可以在根目录给通用规则，在前端、后端或基础设施目录给局部规则。

建议写：

- 项目和目录职责；
- 安装、测试、构建、格式化、迁移和启动命令；
- 代码模式与禁止模式；
- 安全、秘密和生产系统边界；
- 变更前后必须执行的检查；
- 什么叫“完成”。

不要写：

- API Key、密码、令牌或个人数据；
- 当前工单的短期细节；
- 大段愿景、历史或所有文档的复制品；
- 与代码现实不一致的理想规则；
- 无法执行或验证的空泛要求。

16.3 项目配置与全局配置

项目 `.codex/config.toml` 用于受信任仓库的 Codex 行为和工具设置；全局配置用于个人跨仓库默认值。两者都不应替代需求、设计或代码文档。

实践建议：先用默认值工作，只有在重复出现同一摩擦时才配置；配置权限时坚持最小授权；对网络、外部服务和危险命令保持显式审批；团队共享的项目配置要和代码评审一起维护。

16.4 Skill 与 Plugin

当一类任务需要稳定步骤、专门知识、参考资料或脚本时，适合做成 Skill，例如“生成并渲染检查 PDF”“执行数据库迁移审查”。当需要把多个 Skills、工具、MCP、Hooks、资源或应用打包安装时，再升级为 Plugin。

判断方法：

- 一条提示词能完成且很少重复：留在提示词；
- 多次重复同一工作流：做 Skill；
- 需要分发一组能力和配置：做 Plugin。

不要为了“看起来平台化”把每个提示词都变成 Skill，也不要为一个 Skill 创建 Plugin。

16.5 MCP Server 与 Connector

MCP 适合提供模型无法从本地仓库或公开知识可靠获得的实时上下文与动作，例如私有文档、工单、源代码托管或企业系统。连接私有数据时，授权范围应与任务匹配，并区分只读查询和外部写操作。

关键规则：

- 能从本地文件读取的内容，不必绕到 MCP；
- 只需要查询时不要授权写入；
- 外部动作必须有清晰目标、输入和确认边界；
- 工具输出仍是外部内容，不能覆盖用户指令；
- 秘密和个人数据只在确有必要且获得授权时传输。

16.6 Hook 与 Automation

Hook 用于生命周期中的机械执行，例如在命令前阻断危险模式、在文件编辑后运行格式检查、记录工具使用。它适合确定性规则，不适合替代架构或风险判断。

Automation 用于定时或后续工作，例如每天检查失败构建、每周汇总依赖安全告警、在约定时间重新验证工单。只有当任务稳定、重复、结果可观察时才自动化；含糊、需要大量判断或可能产生外部副作用的工作应保留人工门禁。

16.7 CLI、IDE、Codex App、Cloud 与浏览器表面

- CLI：适合终端优先、本地仓库、命令和脚本密集的工作。
- IDE 扩展：适合围绕当前编辑器文件、选区和即时反馈进行修改。
- Codex App：适合桌面环境中的规划、审查、交互式工作和多个工具协作。
- Cloud/Web 工作：适合托管、并行或离线执行；使用前应确认仓库访问、秘密和网络策略。
- 内置浏览器控制：适合由 Codex 验证网页和本地应用的可见行为。
- Chrome 附加控制：适合必须使用用户现有 Chrome 登录状态、标签页或扩展的任务。
- 桌面/Computer Use：适合需要操作本地 GUI 或操作系统界面的工作；风险和确认边界高于纯代码修改。

不要根据“哪个更强”选择表面，而要根据上下文在哪里、需要什么权限、结果如何验证以及外部副作用有多大来选择。

17. 高效使用 Codex 的工作技巧

下面的技巧都包含一个动作和它解决的问题。不要一次全部采用；先选择最能缩短当前反馈回路的三到五条。

17.1 用“目标-上下文-范围-验证”代替长篇角色扮演

告诉 Codex 要实现什么、相关事实在哪里、允许修改什么、如何证明完成。比“你是一位世界级专家”更能约束实际行为，因为工程质量取决于信息和反馈，不取决于称号。

17.2 先要求调查报告，再要求方案

把“找事实”和“做决定”分开。Codex 在还不了解调用链时给出的架构建议通常带有隐含假设；先要求文件、符号、命令和复现证据，可以显著减少幻觉式设计。

17.3 一个提示词只推进一个决策层级

不要同时要求澄清需求、设计数据库、实现 API、写前端和部署。每轮只完成需求、调查、设计、计划或一个实现批次，使每个输出都能被检查和纠正。

17.4 把“不做什么”写进提示词

明确“不升级依赖、不修改公共协议、不重构无关模块、不访问生产”。Codex 会主动寻找完成路径，缺少非目标时，容易把合理但未授权的改进带入变更。

17.5 给出真实命令，不要只说“运行测试”

写出 `pytest tests/auth -q`、`pnpm test`、`go test ./...` 或项目实际命令。真实命令减少工具猜测，也让完成声明有统一证据标准。

17.6 让 Codex 报告退出码、失败数和范围

要求“报告运行了什么、退出码、通过/失败数量、哪些未运行”。这能防止把局部测试通过外推为整个系统通过，也方便下一位工程师复查。

17.7 大任务先做文件地图

在计划前列出将创建/修改的文件及职责。文件地图能暴露组件边界混乱和单文件过载，也能防止实现过程中无控制地扩散。

17.8 控制单批 diff 大小

每一批只解决一个行为或一个紧密组件，完成后检查 diff。小 diff 更容易定位失败、审查安全影响和回滚；大型“一次完成”会把多个因果关系混在一起。

17.9 重新读取即将编辑的文件

对话变长后，Codex 可能依据旧版本上下文。编辑前重新读取文件和相关调用者，能保护用户刚刚做的改动，并降低基于过期假设写补丁的概率。

17.10 对脏工作区先做归属判断

先列出已有改动并区分当前任务与用户工作，不要使用重置或覆盖命令清理。脏工作区不是错误，但任何不明归属的改动都应被视为需要保护的数据。

17.11 先复用，再增加依赖

按“已有代码 -> 标准库 -> 平台原生能力 -> 已安装依赖 -> 新依赖”的顺序选择。每个新依赖都带来供应链、版本、许可证、体积和升级成本。

17.12 对缺陷要求“调用者清单”

让 Codex 搜索即将修改函数的所有调用者。根因修复通常应放在所有路径共享的边界；只在工单提到的页面加保护，会让兄弟路径继续出错。

17.13 用一个强回归测试保护根因

非平凡缺陷至少留下一个能在修复撤销后失败的测试。一个有区分力的回归测试比大量只断言实现细节的测试更能防止复发。

17.14 把浏览器当 UI 变更的编译器

组件测试和类型检查无法发现文本裁切、层级遮挡、焦点丢失、真实网络错误和响应式问题。UI 改动完成后，必须打开页面走关键路径并观察可见状态。

17.15 对数据库变更采用“扩展-迁移-收缩”

先添加兼容结构，再迁移/回填和切换读写，最后在确认旧版本退出后删除旧结构。这样部署和数据库变更可以跨版本共存，回滚不会立刻被结构破坏。

17.16 秘密只通过受控渠道注入

不要把生产凭据粘进提示词、日志或测试固定值。让 Codex 使用环境变量名、秘密管理器引用和脱敏样例，因为对话和命令输出都可能被保存、分享或进入调试记录。

17.17 对破坏性命令要求“先展示目标”

在删除、移动、迁移、重建或覆盖前，先解析并打印绝对目标、影响数量和回滚方式，再单独确认执行。路径错误比命令语法错误更危险。

17.18 中断前生成交接快照

让 Codex 记录目标、已完成步骤、修改文件、验证证据、当前失败、未决决策和下一条命令。新线程应从这个可验证快照继续，而不是依赖对长对话的模糊记忆。

17.19 上下文污染时主动开新线程

当目标已经改变、旧方案被推翻、对话充满无关日志，或 Codex 反复混淆旧状态时，保存交接快照后新开线程。更长上下文不总是更好，干净边界有助于重新建立事实。

17.20 评审提示词要求“找问题”，不要要求“评价质量”

“请审查会导致错误行为、权限绕过、数据损坏、兼容破坏和测试盲区的问题”比“看看代码写得怎么样”更有方向。评审发现应包含触发场景和后果，避免风格噪声。

17.21 把成功声明改成证据请求

不要问“完成了吗”，而问“哪条命令证明了哪项要求，完整输出是什么，还有什么没运行”。问题形式会直接改变 Codex 的停止条件。

17.22 为高风险任务设置明确暂停点

在访问外部系统、安装依赖、运行迁移、删除数据、改变权限和发布前，要求 Codex 停止并请求确认。暂停点把不可逆风险从自动流程中分离出来。

17.23 只把稳定规则写入持久指令

某条规则至少重复出现数次、适用于整个作用域且能清晰执行时，再写入 `AGENTS.md`、Skill 或 Hook。过早持久化会把临时偏好变成长期摩擦。

17.24 用“升级触发条件”控制未来设计

如果当前选择简单实现，记录何时才需要复杂方案，例如“当单节点锁等待超过阈值时改为分区锁”。这既避免过度工程，也让未来扩展有可测触发点。

18. 高频失败模式与纠偏方法

症状	根因	纠偏动作	预防规则
一条提示词要求生成整个应用	没有阶段反馈，假设和错误一起放大	回到需求、调查、设计、计划，按可运行增量重做	一轮只推进一个决策层级
未调查仓库就开始实现	依赖模型对常见项目结构的猜测	停止修改，定位入口、调用链、测试和现有模式	方案前必须有证据化调查
验收标准含糊	把愿望当行为合同	写可观察输入、输出、错误和权限场景	每条标准必须能判断通过/失败
把整个仓库和全部日志塞入上下文	重要信号被噪声淹没	先搜索入口，再按调用链逐层加载	上下文按任务渐进展开
diff 巨大且无法审查	任务没有按行为和依赖拆分	停止继续，拆成独立批次并分别验证	一个批次只解释一个主要因果变化
Codex 说测试通过但没有输出	把模型陈述当成验证证据	重新运行完整命令，报告退出码和失败数	无新鲜命令证据不得声明完成
只运行一个测试就声称全套通过	局部结果被不当外推	分层运行相关、模块和全套检查	报告明确区分已运行与未运行范围
在页面入口加保护但其他路径仍失败	修补症状，没有搜索共享根因	列出所有调用者，在共同边界修复	缺陷修改前必须搜索调用者
为一个实现创建多层接口和工厂	为假想未来提前抽象	回到当前用例，删除未被多个消费者证明的层	第二个真实用例出现后再抽象
顺便升级大量依赖	把功能需求和供应链变化混在一起	撤回无关升级，独立建立升级任务	依赖变化必须有明确原因和单独验证
密钥出现在提示词、日志或提交中	把对话和调试输出误当安全渠道	立即撤销/轮换，清理记录，改用秘密管理	只使用变量名和脱敏样例
使用删除、重置或覆盖命令后丢失用户改动	未确认目标和归属	停止操作，从备份/历史恢复并审计影响	危险操作先展示绝对目标并单独确认
数据迁移无法回滚	直接修改/删除旧结构，忽略版本共存	采用扩展-迁移-收缩，先演练恢复	迁移必须有校验、备份和回滚证据
UI 测试通过但页面裁切或交互失效	只验证代码，没有验证真实渲染	在目标浏览器走关键路径并截图检查	UI 变更必须浏览器验收
发布后发现问题却不能快速恢复	回滚方案只写在文档中未演练	在代表性环境执行回滚/降级演练	未验证回滚不得进入高风险发布
失败测试反复重跑偶尔变绿	用重跑掩盖不稳定根因	固定时间、随机、并发和外部依赖，找波动源	flaky 测试必须登记和处理，不能视为通过
Codex 修改了用户正在做的无关文件	未检查工作区或文件上下文过期	恢复归属清楚的改动，缩小允许文件范围	开始和每批编辑前检查状态与文件
安全扫描有很多结果但没人知道是否阻塞	工具输出没有结合真实可利用性	按证据、攻击路径、影响和可达性分类	扫描是输入，不是最终风险判断
线上问题日志不足，无法定位版本和请求	可观测性留到发布后	增加结构化字段、指标和追踪关联	设计阶段定义关键失败如何被发现
自动化任务持续产生错误外部操作	把需要判断的工作当机械流程	暂停自动化，收紧权限和确认门禁	只自动化稳定、可观察、可回退的步骤

纠偏的统一动作

当你发现工作流已经偏航，不要继续堆提示词。使用以下恢复顺序：

停止写入 -> 保存当前证据 -> 重新确认目标和范围 ->
重建仓库事实 -> 修订设计/计划 -> 从最小失败检查重新开始

如果偏航涉及生产、权限、秘密或数据破坏，先控制影响和保全证据，再讨论代码修复。

19. 可复制的提示词模板

模板不是咒语。替换方括号内容，删除无关字段，并把真实命令和边界写清楚。所有模板都包含目标、上下文、范围、约束、验证、禁止动作和输出格式。

19.1 仓库调查模板

目标：为[功能/缺陷]建立仓库事实，不修改文件。
上下文：[需求、现象、日志或工单]
范围：[目录/服务/分支]
约束：[只读、不可访问的系统、时间限制]
验证命令：[允许运行的搜索、测试或复现命令]
禁止动作：编辑、安装依赖、网络写入、Git 重置、生产访问。

请输出：

1. 适用的 AGENTS.md 和仓库规则；
2. 入口、调用链、数据流、外部依赖；
3. 相关文件、符号和现有测试；
4. 当前行为的复现证据；
5. 事实、假设、未知项；
6. 受影响范围和下一步建议。

19.2 需求澄清模板

目标：把原始需求转换为可实现、可验收的任务，不设计或实现。
上下文：[用户问题和业务目标]
范围：[必须包含]
约束：[兼容、安全、法规、时间、资源]
验证方式：[测试/浏览器/人工验收]
禁止动作：假设未给出的业务规则、扩大范围、开始写代码。

请输出：目标复述、非目标、事实/假设/未知、最多 5 个关键问题、正常/边界/错误/权限场景，以及每条可判断通过失败的验收标准。

19.3 方案比较模板

目标：比较[问题]的 2-3 个真实可行方案，并推荐最小可行方案。
上下文：[调查结果、现有架构]
范围：[组件和接口]
约束：[兼容、性能、安全、数据、团队能力]
验证要求：[原型/测试/压测/迁移演练]
禁止动作：实现代码、只给一个方案、为假想未来过度设计。

请按变更范围、复杂度、运行成本、兼容性、安全、迁移、测试、可观测性、回滚和长期维护对比。推荐方案需给出组件职责、接口、数据流、失败路径、已知限制和升级触发条件。

19.4 实施计划模板

目标：把已批准设计拆成可执行的小步计划，不修改代码。

上下文：[设计文档路径和摘要]

范围：[本轮交付]

约束：[技术栈、依赖、提交和评审要求]

验证命令：[测试、构建、静态检查、浏览器]

禁止动作：TBD、TODO、无精确文件的步骤、未请求重构和依赖升级。

请先列文件结构与职责，再按依赖顺序列任务。每步包含精确文件、动作、必要代码/配置、命令、期望结果和人工门禁。最后核对需求覆盖、占位符和命名一致性。

19.5 功能实现模板

目标：实现计划中的[当前任务]。

上下文：[计划路径、当前失败测试、相关设计]

范围：只允许修改[文件列表]

约束：[API 兼容、性能、安全、风格]

验证命令：[从最小到完整的命令]

禁止动作：扩大范围、升级依赖、修改无关文件、访问生产、清理用户改动。

请重新读取相关文件和调用者，完成最小实现；每次修改后运行相应检查；最后检查 diff。若计划需要改变，停止并报告原因、影响和建议修订，不要自行偏航。

19.6 系统化调试模板

目标：定位并修复[缺陷]的根因，而不是缓解症状。

上下文：复现步骤[]；预期[]；实际[]；日志/错误[]。

范围：[允许调查和修改的模块]

约束：[不可修改的协议、数据或外部系统]

验证命令：[原始复现、相关测试、完整回归]

禁止动作：未复现先修复、同时改多个假设、重跑掩盖 flaky、生产写操作。

请先建立稳定复现，沿调用链收集证据，列出按可能性排序的假设，一次验证一个。找到共享根因和全部调用者后做最小修复，并报告红绿证据、回归结果、剩余风险和无法验证的部分。

19.7 安全审查模板

目标：对[变更]执行基于真实数据流的安全审查。

上下文：[身份模型、数据分类、入口、外部依赖]

范围：[代码、配置、迁移、基础设施]

约束：[合规、数据驻留、秘密策略]

验证命令：[测试、扫描、攻击场景或配置检查]

禁止动作：输出秘密、攻击生产、把扫描数量直接当严重度。

检查认证、授权、对象级越权、输入、输出、秘密、注入、SSRF、重放、并发、幂等、事务、日志泄露、依赖和配置。每个发现给出证据位置、触发场景、影响、严重度、修复、验证方法，并区分事实与假设。

19.8 代码评审模板

目标：判断当前 diff 是否满足需求并可安全合并。

上下文：[需求、设计、计划、diff]

范围：[提交/文件]

约束：[兼容、安全、性能、代码所有权]

验证证据：[实际运行命令及结果]

禁止动作：修改代码、摘要式表扬、只提风格偏好、忽略测试盲区。

优先找错误行为、权限绕过、数据损坏、并发、兼容破坏、资源泄漏、不可运维和测试缺口。按严重度输出；每项包含文件位置、触发条件、具体后果和建议修复。无发现时说明审查范围和剩余风险。

19.9 发布就绪模板

目标：评估[版本/变更]是否具备生产发布条件，不执行生产操作。

上下文：[CI、制品、环境、迁移、发布窗口]

范围：[服务、区域、租户或流量]

约束：[SLO、合规、维护窗口、审批人]

验证命令：[CI、健康检查、迁移演练、回滚演练]

禁止动作：生产写入、访问密钥、扩大流量、执行迁移或删除数据。

请输出发布前/中/后清单、制品与配置校验、迁移顺序、灰度步骤、指标与阈值、停止条件、回滚/降级、负责人和沟通模板。明确所有需人工授权的步骤，并列出生尚未满足的发布门禁。

19.10 中断与交接模板

目标：把当前任务整理成可在新线程或由另一位工程师继续的交接快照。

上下文：[任务和当前阶段]

范围：[只总结当前任务]

约束：[不得猜测未验证状态]

验证命令：[最后运行的命令]

禁止动作：继续实现、把计划项当已完成、遗漏失败和未提交改动。

请输出：目标与范围、已确认决策、完成步骤、修改文件、最新验证证据、当前失败/阻塞、工作区状态、剩余风险、下一条精确命令、需要人工决定的事项。

19.11 提示词自检

发送复杂任务前快速检查：

- 目标是否能用一句话说明？
 - 上下文是事实还是猜测？
 - 允许和禁止范围是否明确？
 - 是否给出真实验证命令？
 - 是否说明外部写操作和生产边界？
 - 是否规定输出格式和停止条件？
 - 如果 Codex 发现范围扩大，是否要求它暂停？
-

附录 A：一页式端到端 workflow

A.1 标准流水线

1. 定义目标

输入：用户问题、业务约束

产物：范围、非目标、验收标准

门禁：关键歧义已确认
2. 准备环境

输入：仓库、AGENTS.md、命令、权限

产物：仓库地图、基线测试、边界

门禁：不覆盖用户改动，环境可运行
3. 调查现状

输入：需求、日志、复现

产物：调用链、事实、根因/扩展点

门禁：结论有代码或命令证据
4. 确认设计

输入：调查和约束

产物：方案比较、数据流、错误与回滚

门禁：人类批准取舍
5. 制定计划

输入：批准设计

产物：文件地图、小步任务、验证命令

门禁：无占位符，覆盖全部需求
6. 建立失败检查

输入：验收标准和风险

产物：失败测试/复现

门禁：因预期缺失行为失败
7. 小批次实现

输入：当前计划任务

产物：最小代码、测试、聚焦 diff

门禁：相关检查通过，无范围漂移
8. 系统验证

输入：完整构建与用户路径

产物：回归、浏览器、集成证据

门禁：无未解释失败
9. 生产风险审查

输入：安全、数据、负载、运维要求

产物：风险处置、迁移、指标、运行手册

门禁：无未处理高危项
10. 代码评审

输入：需求、设计、diff、验证证据

产物：发现、修复、合并决定

门禁：阻塞项关闭
11. 受控发布

输入：制品、迁移、灰度、回滚

产物：发布记录、观测证据

门禁：人工授权，阈值正常，可回滚
12. 反馈沉淀

输入：指标、错误、用户反馈

产物：行动项、测试、文档、持久规则

门禁：观察完成，异常可追踪

A.2 每轮与 Codex 协作的五句话

1. 这一步的目标是什么。
 2. 已确认的事实和相关位置是什么。
 3. 允许修改与禁止修改什么。
 4. 用哪些命令或操作证明完成。
 5. 遇到什么情况必须停止并请求决定。
-

附录 B：生产就绪检查清单

B.1 需求与设计

- ☐ 用户问题、业务目标和成功指标明确。
- ☐ 范围、非目标和兼容要求已确认。
- ☐ 正常、边界、错误和权限场景有验收标准。
- ☐ 风险级别与流程强度匹配。
- ☐ 方案比较涵盖安全、数据、性能、运维和回滚。
- ☐ 设计边界清晰，没有无依据的未来抽象。

B.2 仓库与实现

- ☐ 适用的 `AGENTS.md` 和项目规则已读取。
- ☐ 开始前记录了基线测试和已有改动。
- ☐ 调查覆盖入口、调用者、数据流、外部依赖和测试。
- ☐ 计划包含精确文件、步骤、命令和通过条件。
- ☐ 非平凡行为有失败检查或可运行复现。
- ☐ 变更按小批次完成，diff 无无关格式化和重构。
- ☐ 没有不必要的新依赖或无边界升级。
- ☐ 没有秘密、调试输出、临时文件或意外大文件。

B.3 正确性与质量

- ☐ 原始缺陷复现从失败变为通过，且修复撤销时测试会失败。
- ☐ 相关测试、模块测试和完整测试范围已明确报告。
- ☐ 类型检查、静态检查和构建通过。
- ☐ 错误路径、超时、重试和部分失败得到处理。
- ☐ 公共接口、配置和数据格式保持兼容，或有迁移方案。
- ☐ UI 在真实浏览器验证关键路径、错误态和响应式布局。
- ☐ 无未解释 flaky 测试或环境差异。

B.4 安全与隐私

- ☐ 身份认证和授权在服务端可信边界执行。
- ☐ 对象级权限无法通过修改 ID 绕过。
- ☐ 外部输入经过类型、长度、格式和业务规则验证。
- ☐ 输出和日志不泄露秘密、个人数据或内部堆栈。
- ☐ 查询、命令、模板和 URL 构造不存在注入路径。
- ☐ 外部请求考虑 SSRF、超时、重试和目标白名单。
- ☐ 敏感操作考虑重放、幂等、并发和审计。
- ☐ 新依赖经过来源、许可证、漏洞和维护性评估。
- ☐ 高危发现已修复或有正式风险接受。

B.5 数据与迁移

- ☐ 迁移采用向前/向后兼容顺序。
- ☐ 大表操作评估锁、时长、日志和资源消耗。
- ☐ 回填可分批、可恢复、可重入并有进度观测。
- ☐ 迁移前有备份或恢复路径，并实际验证。
- ☐ 数据校验包含数量、约束、业务不变量和抽样。
- ☐ 回滚不会因旧结构提前删除而失败。
- ☐ 多版本部署期间读写语义一致。

B.6 性能与可靠性

- ☐ 关键查询没有 N+1、无界扫描或缺失索引风险。
- ☐ 内存、文件、连接、线程和队列有界且能释放。
- ☐ 超时、重试、退避和熔断不会形成放大风暴。
- ☐ 缓存失效、一致性和降级行为明确。
- ☐ 容量假设有测量或合理基线。
- ☐ 单点失败和依赖不可用有可接受行为。
- ☐ 关键操作具备幂等或去重保护。

B.7 可观测性与运行

- ☐ 日志包含版本、环境、请求/追踪标识和关键业务维度。
- ☐ 日志级别合理，不记录秘密和完整敏感载荷。
- ☐ 指标覆盖成功率、延迟、吞吐、资源和关键业务结果。
- ☐ 告警对应可操作故障，有阈值、负责人和运行手册。
- ☐ 健康检查区分进程存活与服务就绪。
- ☐ 故障能够定位到版本、区域、依赖和请求路径。

B.8 发布与回滚

- ☐ CI 使用可重复环境运行测试、检查和构建。
- ☐ 制品可追溯到源代码版本且不含环境秘密。
- ☐ 配置和功能开关有默认值、所有者和清理计划。
- ☐ 发布步骤、负责人和通信渠道明确。
- ☐ 灰度范围、观察窗口、成功阈值和停止条件明确。
- ☐ 迁移、部署、健康检查和回滚在代表性环境演练。
- ☐ 生产写操作和危险命令需要明确人工授权。
- ☐ 发布后验证与长期指标观察已经安排。

B.9 评审与知识

- ☐ 评审核对需求、设计、实现、验证和发布，而非只看风格。
- ☐ 所有阻塞问题关闭并重新验证。

- ☐ 已知限制和风险记录在可发现位置。
 - ☐ 运行手册、架构决策、测试和文档已更新。
 - ☐ 稳定的新规则已进入 [AGENTS.md](#)、Skill 或 Hook。
 - ☐ 临时状态和过期指令没有被永久化。
-

附录 C: AGENTS.md 通用模板

Repository Guide

Project overview

- Purpose: [项目解决什么问题]
- Main components: [组件与职责]
- Trust boundaries: [外部输入、敏感系统和生产边界]

Repository map

- `src/...`: [职责]
- `tests/...`: [职责]
- `infra/...`: [职责]

Commands

- Install: `[命令]`
- Run locally: `[命令]`
- Relevant tests: `[命令]`
- Full tests: `[命令]`
- Type/static checks: `[命令]`
- Build: `[命令]`
- Format/lint: `[命令]`
- Database migration check: `[命令]`

Engineering conventions

- Follow existing patterns before adding abstractions.
- Keep changes scoped to the task and preserve unrelated user edits.
- Public API or schema changes require explicit approval and migration notes.
- New dependencies require justification and focused verification.

Testing expectations

- Non-trivial behavior needs a runnable regression check.
- Bug fixes must demonstrate failure before and success after the fix.
- Report exact commands, exit codes, failures, and checks not run.
- UI changes require real browser verification.

Security and data boundaries

- Never print, commit, or paste secrets or production data.
- Never access production or perform external writes without explicit approval.
- Destructive commands require resolved absolute targets and confirmation.
- Authentication, authorization, sensitive data, and migrations are high risk.

Definition of done

- Acceptance criteria are met and mapped to evidence.
- Relevant and full verification appropriate to risk has passed.
- Diff contains no unrelated changes, debug code, secrets, or artifacts.
- Security, migration, observability, release, and rollback are addressed.
- Documentation and runbooks are updated where behavior changed.

使用提示：根 **AGENTS.md** 保留全局规则；只有子目录确有不同命令或约定时才增加嵌套文件。定期删除与代码现实不一致的内容。

附录 D：能力选择速查表

你遇到的情况	首选做法	升级条件
一次任务有特殊范围	当前提示词	不需要升级
每次都要提醒同一仓库命令	AGENTS.md	子树规则不同则加嵌套文件
需要团队共享 Codex 工具设置	项目配置	需管理员策略时交由工作区管理
多仓库重复同一流程	Skill	需要一起分发工具/资源时做 Plugin
需要私有或实时外部信息	MCP/Connector	需要写操作时增加授权和确认
必须机械阻断某类命令	Hook	规则需要判断时保留人工评审
需要定期检查或提醒	Automation	外部副作用高时保留人工门禁
本地终端与脚本密集	CLI	需要编辑器上下文时用 IDE
需要桌面规划、审查和多工具协作	Codex App	需要托管并行执行时评估 Cloud
需要验证网页可见行为	内置浏览器	需要用户登录态时用 Chrome 表面
需要操作本地 GUI	Computer Use	先确认权限和外部副作用

D.1 不要升级机制的信号

- 规则只会使用一次；
- 内容仍在频繁变化；
- 需要人类业务判断，无法机械执行；
- 权限成本高于节省的时间；
- 失败结果不容易观察；
- 一条短提示词已经足够。

附录 E：术语与官方参考资料

E.1 术语表

术语	本手册中的含义
验收标准	能通过测试、命令、界面操作或人工检查判断通过/失败的行为条件
退出条件	一个阶段允许进入下一阶段前必须满足的可观察条件
人工门禁	必须由有责任 and 权限的人确认的决策点
证据	刚刚运行的命令输出、可见界面状态、数据校验或审查记录
回归测试	能阻止已修复行为再次出现的检查
爆炸半径	变更失败时可能影响的用户、数据、服务和时间范围
可逆性	变更能否安全、快速恢复到之前状态
Worktree	与同一 Git 仓库关联的隔离工作目录
AGENTS.md	随仓库保存、按目录作用域向 Codex 提供工程指导的文件
Skill	可复用任务工作流，可包含说明、参考资料和脚本
Plugin	可安装的 Codex 能力包，可组合 Skills、工具、配置和资源
MCP	让模型连接外部数据和动作的标准化工具接口
Hook	在工具或生命周期事件周围执行的机械规则
Automation	按计划或后续触发的检查、提醒和工作

E.2 官方参考资料

以下为 OpenAI 官方 Codex 文档入口。页面结构和功能可能更新，使用时以页面当前内容和所在组织的管理员策略为准。

- Codex 文档首页：<https://developers.openai.com/codex/>
- Codex 自定义与 AGENTS.md：<https://developers.openai.com/codex/concepts/customization#agents-guidance>
- Codex Skills：<https://developers.openai.com/codex/concepts/customization#skills>
- Codex MCP：<https://developers.openai.com/codex/concepts/customization#mcp>
- Plugin 结构：<https://developers.openai.com/codex/plugins/build#plugin-structure>
- 高级配置与 Hooks：<https://developers.openai.com/codex/config-advanced#hooks>
- Automations：<https://developers.openai.com/codex/app/automations#thread-automations>
- config.toml 配置参考：<https://developers.openai.com/codex/config-reference#configtoml>

E.3 最后的工作原则

先理解，再修改。
先定义证据，再宣称完成。
让任务足够小，使失败能够定位。
让权限足够窄，使错误不能扩大。
让发布可观察，使异常能够停止。
让变更可回滚，使试错不会变成事故。

Codex 能显著压缩阅读、实现和验证所需的时间，但不会替代工程责任。最好的工作流不是让模型写最多代码，而是让每个重要结论都有证据、每个高风险动作都有边界、每次发布都有退路。